

AAIS Cognitive Bridge Module — Codex Pack

This pack contains three aligned artifacts for the **AAIS Cognitive Bridge Module**:

1. **Codex-Ready Implementation Spec**
 2. **Canonical AAIS Markdown Doc**
 3. **JSON Schema Pack**
-

1) Codex-Ready Implementation Spec

Title

Implement `AAIS-CBM-01` — Cognitive Bridge Module

Objective

Build a governed translation module that converts operator intent, subsystem identity, and mythic framing into bounded doctrine bindings, invariant activation sets, drift detection signals, and audit-ready governance artifacts.

Build Goal

Implement a production-safe, testable Python module for AAIS that:

- resolves operator intent into a governed intent packet
- binds doctrine and invariants to the current runtime context
- maps approved mythic terms into operational semantics
- detects identity and cross-lane drift
- emits governance packets for downstream enforcement layers
- records audit events for all key decisions and corrections

Non-Goals

Do **not**:

- create new law dynamically
- let mythic framing alter runtime truth
- replace Jarvis authority
- let the bridge execute actions directly
- allow unmapped symbolic language to change runtime state

Required Constraints

- Fail closed on identity or law violations
- Fail safe on mythic ambiguity
- Be deterministic where possible
- Use explicit registries for doctrine, invariants, and mythic mappings
- Emit structured outputs only; no hidden side effects
- Make all correction actions auditable

Deliverables

Create the following files:

```
src/cognitive_bridge/__init__.py
src/cognitive_bridge/models.py
src/cognitive_bridge/exceptions.py
src/cognitive_bridge/registry.py
src/cognitive_bridge/policies.py
src/cognitive_bridge/intent_resolver.py
src/cognitive_bridge/doctrine_binder.py
src/cognitive_bridge/mythic_mapper.py
src/cognitive_bridge/drift_monitor.py
src/cognitive_bridge/governance_emitter.py
src/cognitive_bridge/audit.py
src/cognitive_bridge/service.py

tests/test_cognitive_bridge_intent.py
tests/test_cognitive_bridge_doctrine.py
tests/test_cognitive_bridge_mythic.py
tests/test_cognitive_bridge_drift.py
tests/test_cognitive_bridge_governance.py
tests/test_cognitive_bridge_audit.py
tests/test_cognitive_bridge_service.py
```

Runtime Contract

The service must expose the following primary API:

```
class CognitiveBridgeService:
    def resolve_intent(self, operator_input: str, runtime_context: dict) ->
    GovernedIntentPacket:
        ...

    def bind_doctrine(self, intent_packet: GovernedIntentPacket) ->
```

```

DoctrineBindingSet:
    ...

    def map_mythic_context(self, text: str) -> MythicMappingResult:
        ...

    def activate_invariants(
        self,
        intent_packet: GovernedIntentPacket,
        doctrine_binding: DoctrineBindingSet,
    ) -> InvariantActivationSet:
        ...

    def detect_drift(
        self,
        subsystem_output: dict,
        runtime_context: dict,
    ) -> list[DriftAlertPacket]:
        ...

    def emit_governance_packet(
        self,
        intent_packet: GovernedIntentPacket,
        doctrine_binding: DoctrineBindingSet,
        invariant_set: InvariantActivationSet,
        mythic_result: MythicMappingResult | None = None,
    ) -> GovernancePacket:
        ...

    def record_audit_event(self, event: AuditEvent) -> None:
        ...

```

Implementation Notes

models.py

Define dataclasses or Pydantic models for:

- GovernedIntentPacket
- DoctrineBindingSet
- InvariantActivationSet
- MythicMappingResult
- DriftAlertPacket
- CorrectionDirective
- GovernancePacket
- AuditEvent

- `OperatorHealthSnapshot`

`registry.py`

Provide immutable or tightly controlled registries for:

- doctrine bindings
- invariants
- mythic translation mappings
- subsystem identity manifests

`intent_resolver.py`

Responsibilities:

- normalize operator requests into a stable `intent_id`
- classify domain, risk, priority, and active modules
- produce doctrine candidates
- degrade safely on unknown intent

Required behavior:

- direct string heuristics are acceptable for v1
- must be testable and deterministic
- unknown intent should return a minimal governed packet with conservative defaults

`doctrine_binder.py`

Responsibilities:

- bind doctrine by `intent_id`, domain, and runtime context
- resolve doctrine conflicts conservatively
- attach audit mode and binding confidence

Required behavior:

- stricter doctrine wins on conflict
- if doctrine is incomplete for a medium/high-risk request, fail closed

`mythic_mapper.py`

Responsibilities:

- parse text for approved symbolic language
- map approved symbols to operational tags/effects
- reject or preserve-only unmapped symbols

Required behavior:

- unmapped symbols must never alter runtime behavior
- return presentation-only flags when no valid mapping exists

`drift_monitor.py`

Responsibilities:

- detect identity leakage
- detect authority borrowing n- detect cross-lane contamination
- detect attempts to override runtime truth with narrative framing

Required behavior:

- use explicit rule checks against manifests and active invariants
- emit structured `DriftAlertPacket` entries

`governance_emitter.py`

Responsibilities:

- assemble final `GovernancePacket`
- include doctrine bindings, invariants, risk posture, audit mode, and correction policy
- produce downstream-consumable structured output

`audit.py`

Responsibilities:

- append audit-safe events to in-memory history for v1
- expose retrieval helpers
- provide reset hooks for tests

`service.py`

Responsibilities:

- orchestrate all internal module calls
- provide a single bounded entry point
- refuse execution when required registries are missing

Required Data Models

GovernedIntentPacket

```
{
  "intent_id": "brindle_hollow_integration",
  "operator_goal": "integrate narrative domain into governed runtime context",
  "domain": "narrative_governance",
  "priority": "high",
  "risk_class": "medium",
  "active_modules": ["jarvis", "nova", "story_forge"],
  "requested_mode": "governed_integration",
  "requires_approval": false,
  "doctrine_candidates": [
    "identity_stability",
    "bounded_adaptation",
    "cross_lane_isolation",
    "canon_truth_preservation"
  ]
}
```

DoctrineBindingSet

```
{
  "binding_id": "bind_001",
  "intent_id": "brindle_hollow_integration",
  "active_doctrine": [
    "identity_stability",
    "bounded_adaptation",
    "cross_lane_isolation",
    "mythic_operational_correspondence"
  ],
  "binding_source": "cognitive_bridge",
  "binding_confidence": 0.96,
  "audit_mode": "elevated"
}
```

InvariantActivationSet

```
{
  "activation_id": "inv_001",
  "intent_id": "brindle_hollow_integration",
  "invariants": [
    "nova_must_not_assume_operator_authority",
  ]
}
```

```

    "story_forge_must_not_override_runtime_truth",
    "jarvis_retains_final_governance_authority",
    "forge_must_remain_non_persona",
    "cross_lane_identity_bleed_is_blocked"
  ],
  "fail_policy": "fail_closed_on_identity_violation",
  "degrade_policy": "degrade_to_safe_summary"
}

```

DriftAlertPacket

```

{
  "drift_id": "drift_014",
  "drift_type": "identity_authority_leakage",
  "source_module": "nova",
  "severity": "high",
  "detected_behavior": "operator-style governance language",
  "violated_invariant": "nova_must_not_assume_operator_authority",
  "recommended_action": "rewrite_or_reroute",
  "timestamp": "2026-04-20T12:00:00Z"
}

```

GovernancePacket

```

{
  "packet_id": "gov_001",
  "intent_id": "brindle_hollow_integration",
  "doctrine_binding": "bind_001",
  "invariant_activation": "inv_001",
  "risk_class": "medium",
  "audit_mode": "elevated",
  "status": "approved_for_governed_execution"
}

```

Acceptance Criteria

Functional

- Known intent resolves to a stable `intent_id`
- Doctrine binding returns the correct doctrine set
- Invariant activation returns the correct required invariants
- Approved mythic symbols map to known operational meanings
- Unmapped symbols do not affect runtime state
- Identity drift is detected for authority leakage and cross-lane bleed

- Governance packet assembly is deterministic and audit-ready

Safety

- Missing doctrine for medium/high-risk paths fails closed
- Missing invariant pack for medium/high-risk paths fails closed
- Unmapped mythic symbols remain presentation-only
- Identity drift produces correction directives

Test

- All new tests pass under pytest
- No network calls required
- Module can run in isolation

Minimum Test Matrix

Intent tests

- known request → stable governed intent
- unknown request → conservative defaults
- risk elevation respected

Doctrine tests

- known intent activates correct doctrine
- doctrine conflicts resolve conservatively
- missing doctrine fails closed when required

Mythic tests

- approved symbol → operational mapping
- unapproved symbol → presentation only
- no runtime mutation from unmapped symbol

Drift tests

- Nova authority leakage detected
- Forge persona bleed detected
- Story Forge truth override attempt detected
- cross-lane signature contamination detected

Governance tests

- governance packet contains active doctrine, invariants, audit mode, and status
- correction policies attach correctly on violation

Audit tests

- every doctrine bind logs an audit event
- every drift event logs an audit event
- reset helpers work for tests

Build Sequence

1. implement models and exceptions
2. implement registries and static seed data
3. implement intent resolver
4. implement doctrine binder
5. implement mythic mapper
6. implement invariant activation
7. implement drift monitor
8. implement governance emitter
9. wire service orchestration
10. add full test coverage

Seed Registry Content

Doctrine seeds

- identity_stability
- bounded_adaptation
- cross_lane_isolation
- canon_truth_preservation
- mythic_operational_correspondence

Invariant seeds

- nova_must_not_assume_operator_authority
- story_forge_must_not_override_runtime_truth
- jarvis_retains_final_governance_authority
- forge_must_remain_non_persona
- cross_lane_identity_bleed_is_blocked

Mythic mapping seeds

- ward -> protective_constraint_boundary
- forge -> isolated_construction_lane
- trial -> verification_checkpoint
- drift -> invariant_deviation
- gate -> approval_boundary
- lawbook -> canonical_doctrine_registry

Definition of Done

The module is done when:

- service entry point works end-to-end
 - test suite passes
 - outputs are deterministic and structured
 - doctrine binding, mythic mapping, drift detection, and governance emission are all auditable
 - the module can be attached to Jarvis/Law Engine integration points without changing its contract
-

2) Canonical AAIS Markdown Doc

AAIS Canonical Module Record

Module

Cognitive Bridge Module

Internal ID

AAIS-CBM-01

Classification

Governance Translation Module

Canonical Definition

The Cognitive Bridge Module is the governed translation layer that converts operator intent, subsystem identity, and mythic framing into bounded doctrine bindings, invariant activation sets, drift detection signals, and audit-ready governance artifacts across AAIS.

Core Function

Maintain coherence between:

- meaning
- identity
- doctrine
- invariants
- runtime truth

Why It Exists

AAIS contains multiple valid but different modes of operation:

- Jarvis as sovereign operator-facing orchestrator
- Nova as bounded companion
- Forge as isolated contractor
- Story Forge as mythic/narrative runtime
- Law Engine and invariant enforcement as truth and execution boundary

The Cognitive Bridge Module exists to prevent drift between those layers.

Canonical Responsibilities

- resolve current operator intent into governed runtime meaning
- bind active doctrine to the current task or context
- map approved mythic language into operational semantics
- detect identity, authority, and cross-lane drift
- emit governance artifacts for downstream enforcement
- record alignment and correction events
- expose doctrine health to the operator

Canonical Non-Responsibilities

- it does not create new law
- it does not replace Jarvis authority
- it does not let mythic language override truth
- it does not mutate subsystem identities
- it does not directly execute downstream actions

Governing Principle

Meaning must become lawfully actionable before it may influence runtime behavior.

Position in AAIS

```
Operator Intent
-> Identity-bearing and intent-bearing systems
-> Cognitive Bridge Module
-> Law / Invariants / Runtime Enforcement
-> Approved Execution + Audit
```

Inputs

- operator request

- runtime context
- identity manifests
- doctrine registry
- invariant registry
- mythic translation registry
- policy state

Outputs

- governed intent packet
- doctrine binding set
- invariant activation set
- drift alert packet
- correction directive
- governance packet
- audit event
- operator health snapshot

Core Invariants

Identity Stability

- Jarvis retains final governance authority.
- Nova remains bounded companion identity.
- Forge remains non-persona and contract-bound.
- Story Forge may not mutate runtime truth.

Cross-Lane Isolation

- No subsystem may borrow authority from another subsystem without explicit law.
- Narrative framing may not rewrite execution constraints.
- Companion adaptation may not become operator control.
- Contractor behavior may not acquire symbolic identity.

Truth Preservation

- Runtime truth precedes presentation.
- Law precedes symbolic expression.
- Verified state precedes mythic rendering.
- Mythic language may enrich approved truth but may not replace it.

Audit Discipline

- Every doctrine bind is traceable.
- Every drift event is recorded.
- Every correction action references a violated rule or policy.
- Hidden doctrine mutation is prohibited.

Mythic Translation Law

No mythic symbol may affect runtime behavior unless it has an approved operational mapping in the mythic translation registry.

Default Safety Posture

- fail closed on identity and law violations
- fail safe on mythic ambiguity
- fail visible on doctrine uncertainty

Success Condition

The module succeeds when operator intent, subsystem identity, mythic framing, and runtime law remain aligned without requiring manual reconciliation by the operator.

Failure Condition

The module fails when symbolic meaning influences runtime state without doctrine binding, when subsystem identities bleed across lanes, or when the operator becomes the manual bridge between meaning and law.

3) JSON Schema Pack

Below are first-pass JSON Schemas for the core contracts.

governed_intent_packet.schema.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "aais/cognitive_bridge/governed_intent_packet.schema.json",
  "title": "GovernedIntentPacket",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "intent_id",
    "operator_goal",
    "domain",
    "priority",
    "risk_class",
    "active_modules",
    "requested_mode",
    "requires_approval",
  ],
}
```

```

    "doctrine_candidates"
  ],
  "properties": {
    "intent_id": { "type": "string", "minLength": 1 },
    "operator_goal": { "type": "string", "minLength": 1 },
    "domain": { "type": "string", "minLength": 1 },
    "priority": { "type": "string", "enum": ["low", "medium", "high",
"critical"] },
    "risk_class": { "type": "string", "enum": ["low", "medium", "high"] },
    "active_modules": {
      "type": "array",
      "minItems": 1,
      "items": { "type": "string", "minLength": 1 },
      "uniqueItems": true
    },
    "requested_mode": { "type": "string", "minLength": 1 },
    "requires_approval": { "type": "boolean" },
    "doctrine_candidates": {
      "type": "array",
      "items": { "type": "string", "minLength": 1 },
      "uniqueItems": true
    }
  }
}

```

doctrine_binding_set.schema.json

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "aais/cognitive_bridge/doctrine_binding_set.schema.json",
  "title": "DoctrineBindingSet",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "binding_id",
    "intent_id",
    "active_doctrine",
    "binding_source",
    "binding_confidence",
    "audit_mode"
  ],
  "properties": {
    "binding_id": { "type": "string", "minLength": 1 },
    "intent_id": { "type": "string", "minLength": 1 },
    "active_doctrine": {
      "type": "array",

```

```

    "minItems": 1,
    "items": { "type": "string", "minLength": 1 },
    "uniqueItems": true
  },
  "binding_source": { "type": "string", "const": "cognitive_bridge" },
  "binding_confidence": { "type": "number", "minimum": 0, "maximum": 1 },
  "audit_mode": { "type": "string", "enum": ["standard", "elevated",
"critical"] }
}
}

```

invariant_activation_set.schema.json

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "aais/cognitive_bridge/invariant_activation_set.schema.json",
  "title": "InvariantActivationSet",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "activation_id",
    "intent_id",
    "invariants",
    "fail_policy",
    "degrade_policy"
  ],
  "properties": {
    "activation_id": { "type": "string", "minLength": 1 },
    "intent_id": { "type": "string", "minLength": 1 },
    "invariants": {
      "type": "array",
      "minItems": 1,
      "items": { "type": "string", "minLength": 1 },
      "uniqueItems": true
    },
    "fail_policy": { "type": "string", "minLength": 1 },
    "degrade_policy": { "type": "string", "minLength": 1 }
  }
}

```

mythic_mapping_result.schema.json

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",

```

```

"$id": "aais/cognitive_bridge/mythic_mapping_result.schema.json",
"title": "MythicMappingResult",
"type": "object",
"additionalProperties": false,
"required": [
  "source_text",
  "mapped_symbols",
  "unmapped_symbols",
  "presentation_only",
  "runtime_tags"
],
"properties": {
  "source_text": { "type": "string" },
  "mapped_symbols": {
    "type": "array",
    "items": {
      "type": "object",
      "additionalProperties": false,
      "required": ["symbol", "maps_to"],
      "properties": {
        "symbol": { "type": "string", "minLength": 1 },
        "maps_to": { "type": "string", "minLength": 1 }
      }
    }
  },
  "unmapped_symbols": {
    "type": "array",
    "items": { "type": "string", "minLength": 1 },
    "uniqueItems": true
  },
  "presentation_only": { "type": "boolean" },
  "runtime_tags": {
    "type": "array",
    "items": { "type": "string", "minLength": 1 },
    "uniqueItems": true
  }
}
}

```

drift_alert_packet.schema.json

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "aais/cognitive_bridge/drift_alert_packet.schema.json",
  "title": "DriftAlertPacket",
  "type": "object",

```



```

"additionalProperties": false,
"required": [
  "drift_id",
  "drift_type",
  "source_module",
  "severity",
  "detected_behavior",
  "violated_invariant",
  "recommended_action",
  "timestamp"
],
"properties": {
  "drift_id": { "type": "string", "minLength": 1 },
  "drift_type": { "type": "string", "minLength": 1 },
  "source_module": { "type": "string", "minLength": 1 },
  "severity": { "type": "string", "enum": ["low", "medium", "high",
"critical"] },
  "detected_behavior": { "type": "string", "minLength": 1 },
  "violated_invariant": { "type": "string", "minLength": 1 },
  "recommended_action": { "type": "string", "minLength": 1 },
  "timestamp": { "type": "string", "format": "date-time" }
}
}

```

correction_directive.schema.json

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "aais/cognitive_bridge/correction_directive.schema.json",
  "title": "CorrectionDirective",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "directive_id",
    "triggered_by",
    "action",
    "target_module",
    "policy",
    "fallback_mode",
    "requires_audit_record"
  ],
  "properties": {
    "directive_id": { "type": "string", "minLength": 1 },
    "triggered_by": { "type": "string", "minLength": 1 },
    "action": { "type": "string", "minLength": 1 },
    "target_module": { "type": "string", "minLength": 1 },

```

```

    "policy": { "type": "string", "minLength": 1 },
    "fallback_mode": { "type": "string", "minLength": 1 },
    "requires_audit_record": { "type": "boolean" }
  }
}

```

governance_packet.schema.json

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "aais/cognitive_bridge/governance_packet.schema.json",
  "title": "GovernancePacket",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "packet_id",
    "intent_id",
    "doctrine_binding",
    "invariant_activation",
    "risk_class",
    "audit_mode",
    "status"
  ],
  "properties": {
    "packet_id": { "type": "string", "minLength": 1 },
    "intent_id": { "type": "string", "minLength": 1 },
    "doctrine_binding": { "type": "string", "minLength": 1 },
    "invariant_activation": { "type": "string", "minLength": 1 },
    "risk_class": { "type": "string", "enum": ["low", "medium", "high"] },
    "audit_mode": { "type": "string", "enum": ["standard", "elevated",
"critical"] },
    "status": { "type": "string", "minLength": 1 }
  }
}

```

audit_event.schema.json

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "aais/cognitive_bridge/audit_event.schema.json",
  "title": "AuditEvent",
  "type": "object",
  "additionalProperties": false,
  "required": [

```

```

    "event_id",
    "event_type",
    "timestamp",
    "severity",
    "details"
  ],
  "properties": {
    "event_id": { "type": "string", "minLength": 1 },
    "event_type": { "type": "string", "minLength": 1 },
    "timestamp": { "type": "string", "format": "date-time" },
    "severity": { "type": "string", "enum": ["info", "warning", "high",
"critical"] },
    "details": {
      "type": "object",
      "additionalProperties": true
    }
  }
}

```

operator_health_snapshot.schema.json

```

{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "aais/cognitive_bridge/operator_health_snapshot.schema.json",
  "title": "OperatorHealthSnapshot",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "snapshot_id",
    "intent_id",
    "doctrine_state",
    "jarvis_state",
    "nova_state",
    "story_forge_state",
    "forge_state",
    "drift_alert_count",
    "active_violations",
    "audit_mode",
    "overall_status"
  ],
  "properties": {
    "snapshot_id": { "type": "string", "minLength": 1 },
    "intent_id": { "type": "string", "minLength": 1 },
    "doctrine_state": { "type": "string", "minLength": 1 },
    "jarvis_state": { "type": "string", "minLength": 1 },
    "nova_state": { "type": "string", "minLength": 1 },

```

```
"story_forge_state": { "type": "string", "minLength": 1 },
"forge_state": { "type": "string", "minLength": 1 },
"drift_alert_count": { "type": "integer", "minimum": 0 },
"active_violations": { "type": "integer", "minimum": 0 },
"audit_mode": { "type": "string", "enum": ["standard", "elevated",
"critical"] },
"overall_status": { "type": "string", "minLength": 1 }
}
}
```

Codex Handoff Note

Use the **Implementation Spec** as the build contract. Use the **Canonical AAIS Markdown Doc** as the doctrine source. Use the **JSON Schema Pack** as the structured contract surface.

If these ever disagree, treat the canonical doc as doctrine, the schemas as enforcement shape, and the implementation spec as execution guidance.